ARM for Cyber Security Students

Tom Chothia

The ARM CPU

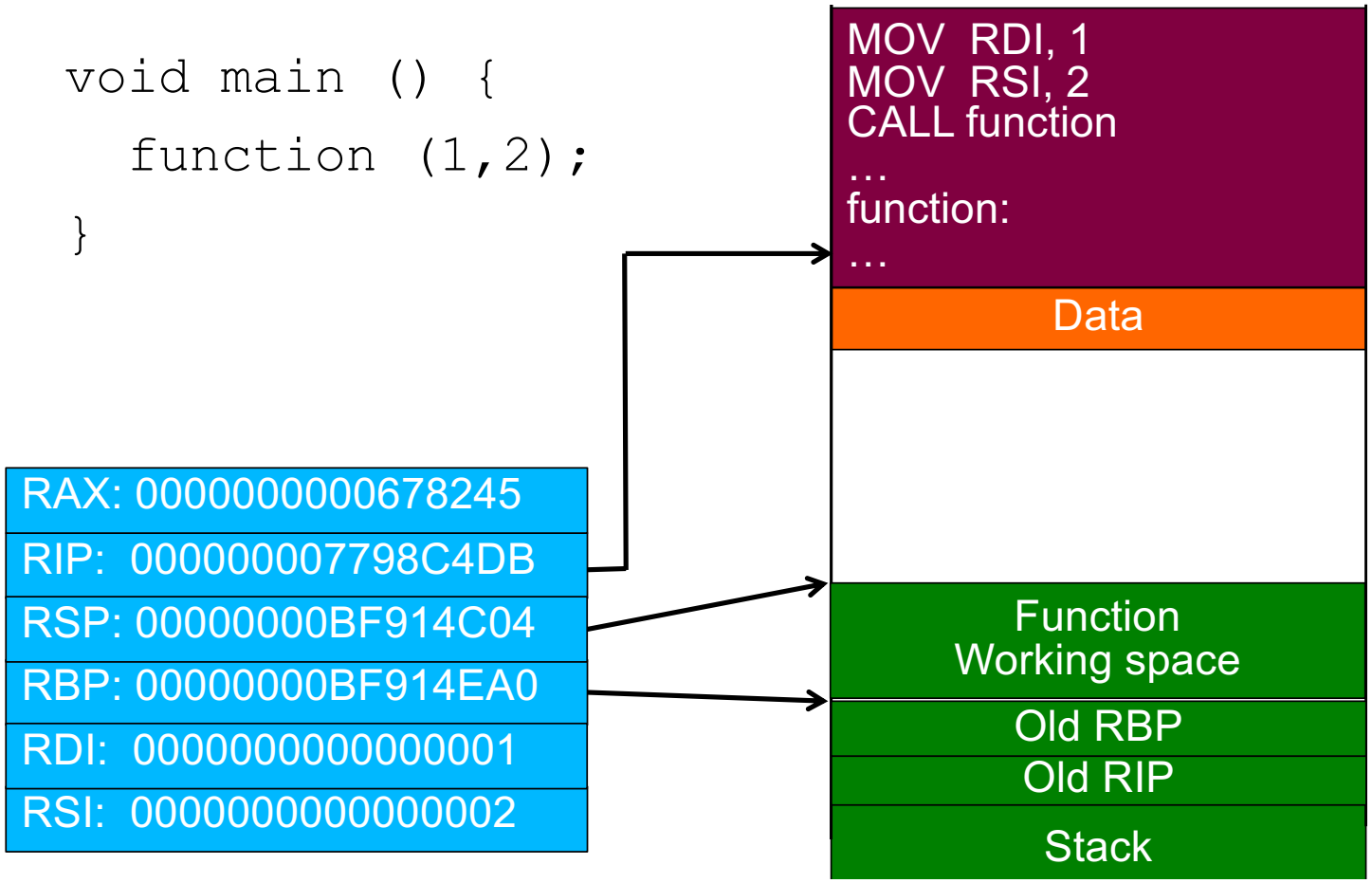
- Developed in Cambridge UK, by Acorn Computers in the 1980.
- A more rational redesign of of the CPU. Fewer commands, uses less power.
- Slowly been gaining popularity since the 1980s.
- Now on ***Macs***, (AArch64, 64-bit) phones, IoT devices (ARM7, 32-bit). Soon windows laptops too.

ARM vs x64

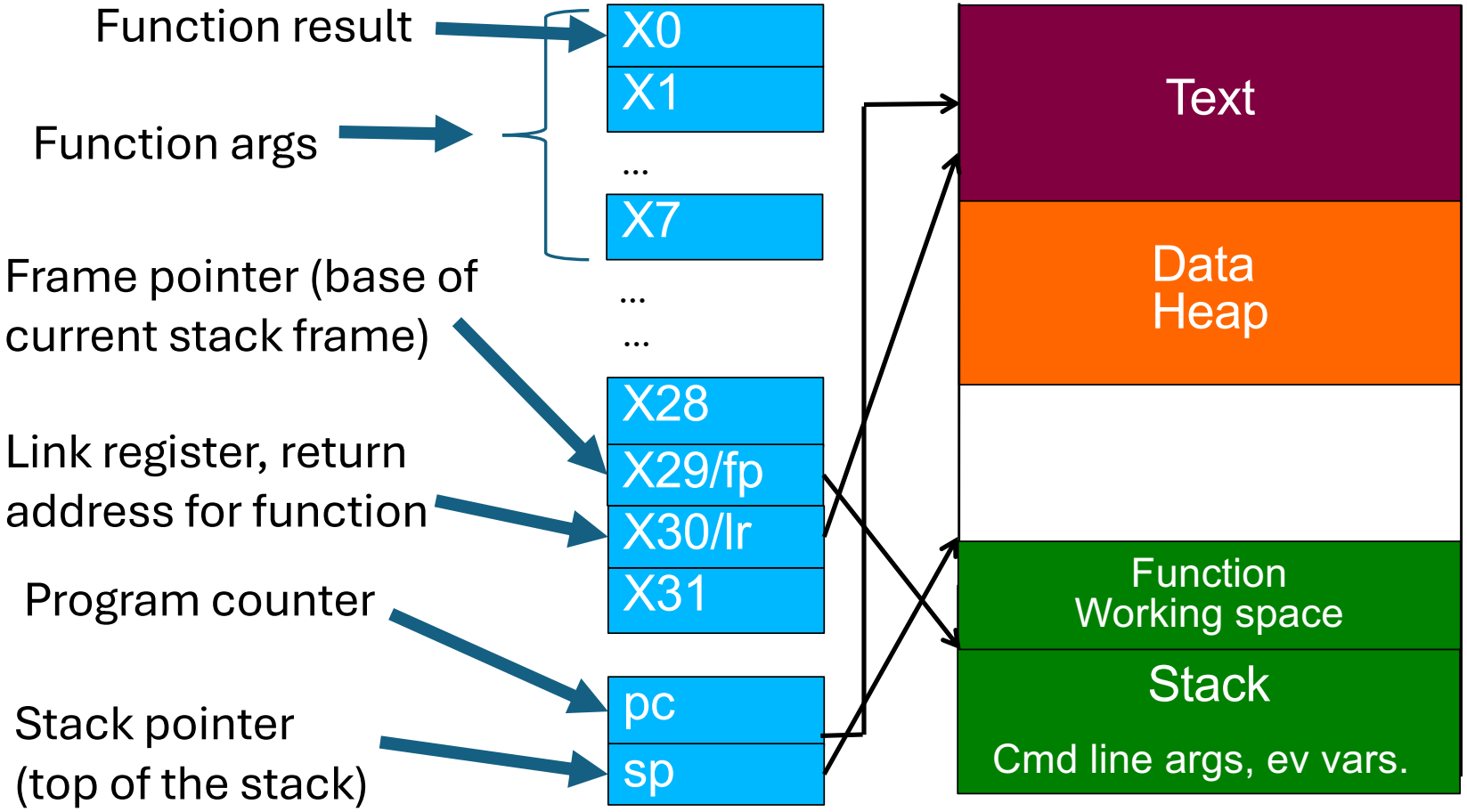
- Similar stack layout and registers to x64.
- Smaller instruction set, e.g., only load and store commands can read and write memory, All commands are 32-bits long.
- More registers. Much more done in registers**,** fewer memory accesses.
- Functions manage the stack themselves (no automatic new stack frame with `call` and `ret` like in x64).
- Function return address goes into **lr**, the link register, (old **lr** needs to be pushed onto the stack). Functions that dont call others, don’t need to do this

The x86-64 Architecture

```
void main () {
   function (1,2);
}
```



ARM AArch 64Architecture



AArch7 ARM Registers

- **X0** to **X31** 64-bit registers. First 32 bits addressed as **W0** to **W31**.
  - **X29** of the frame pointer (**FP**) - points to base of current stack frame (like RBP in x64).
  - **X30** is the link register (**LR**) stores the return address of the current functions (in x64 this is put on the stack).
    - Before calling a function, LR must be stored on the stack.

- Stack pointer (**sp**) register, points to top of stack (like RSP in x64).

- Program counter (**pc**) register, points to next command to execute (like RIP in x64).

- **wzr** a register that is always zero (x64 often does, e.g., xor rax rax to get 0)

Branches and calls

- In x64 we have J for jump, but in ARM we have b for branch

```
b               LAB_100003d94
```

Means branch (jump) to location LAB_100003d94

Branch to link bl is used to call functions (like call in x64)

```
bl functionName
```

Means store program counter in LR register and jump to functionName.

Conditional branches

if x1=x2 then goto label.

```
cmp x1 x2
bz label
```

Calculate x1-x2 and set flags

Branch if zero flag set

```
sub x3 x1 x2
cbz x3 label
```

x3=x1-x2

Branch if x3=0

```
subs x3 x1 x2
bz label
```

Calculate x3=x1-x2 and set flags

Branch if zero flag set

```
sub x3 x1 x2
cset x3, z
tbz x3,0, label
```

x3=x1-x2

Set x3 to 1 if its currently zero

Branch is bit 0 Of x3 is zero

Calling Convention/ Functions in ARM

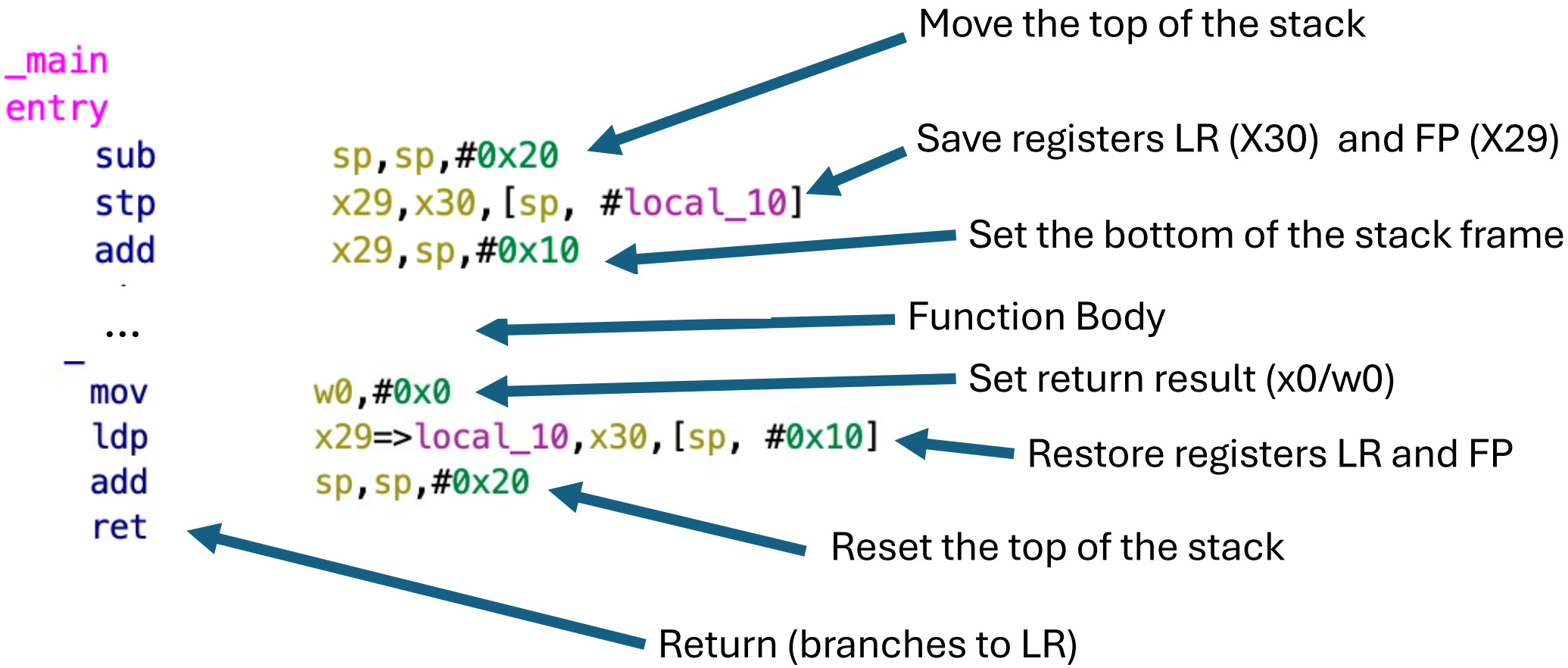
- Functions arguments go in X0 to X7, then on the stack.
  - If a function has optional arguments (like printf) these are always on the stack.

- Function is called, with BL <functionName>, this stores the pc in the lr register.
  - The caller must store the old lr on the stack.

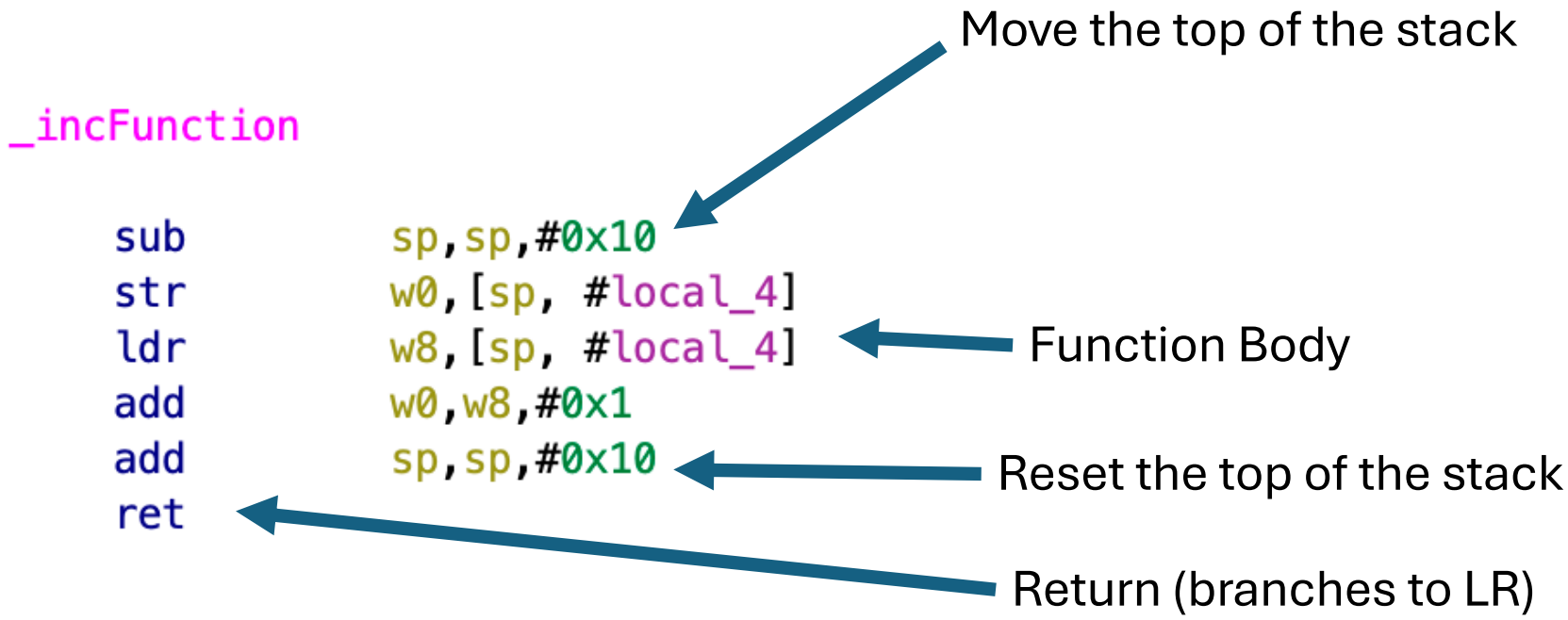
- Callee may set sp and fp to make a new stack frame.

- Return with ret (this just moves lr to pc)

General Function Layout



Leaf Functions Don’t need to Save LR

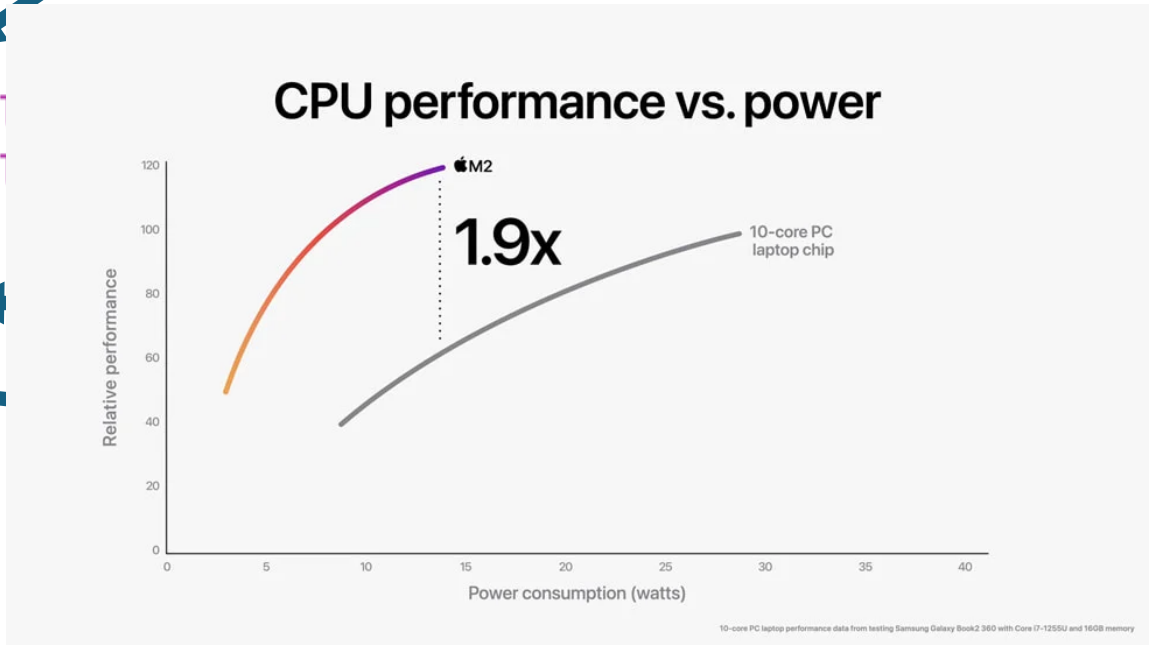


Leaf Functions Don’t need to Save LR

Move the top of the stack

```
_incFunction

    sub         sp,sp,#0x10
    str         w0,[sp, #loca
    ldr         w8,[sp, #loca
    add         w0,w8,#0x1
    add         sp,sp,#0x10
    ret
```



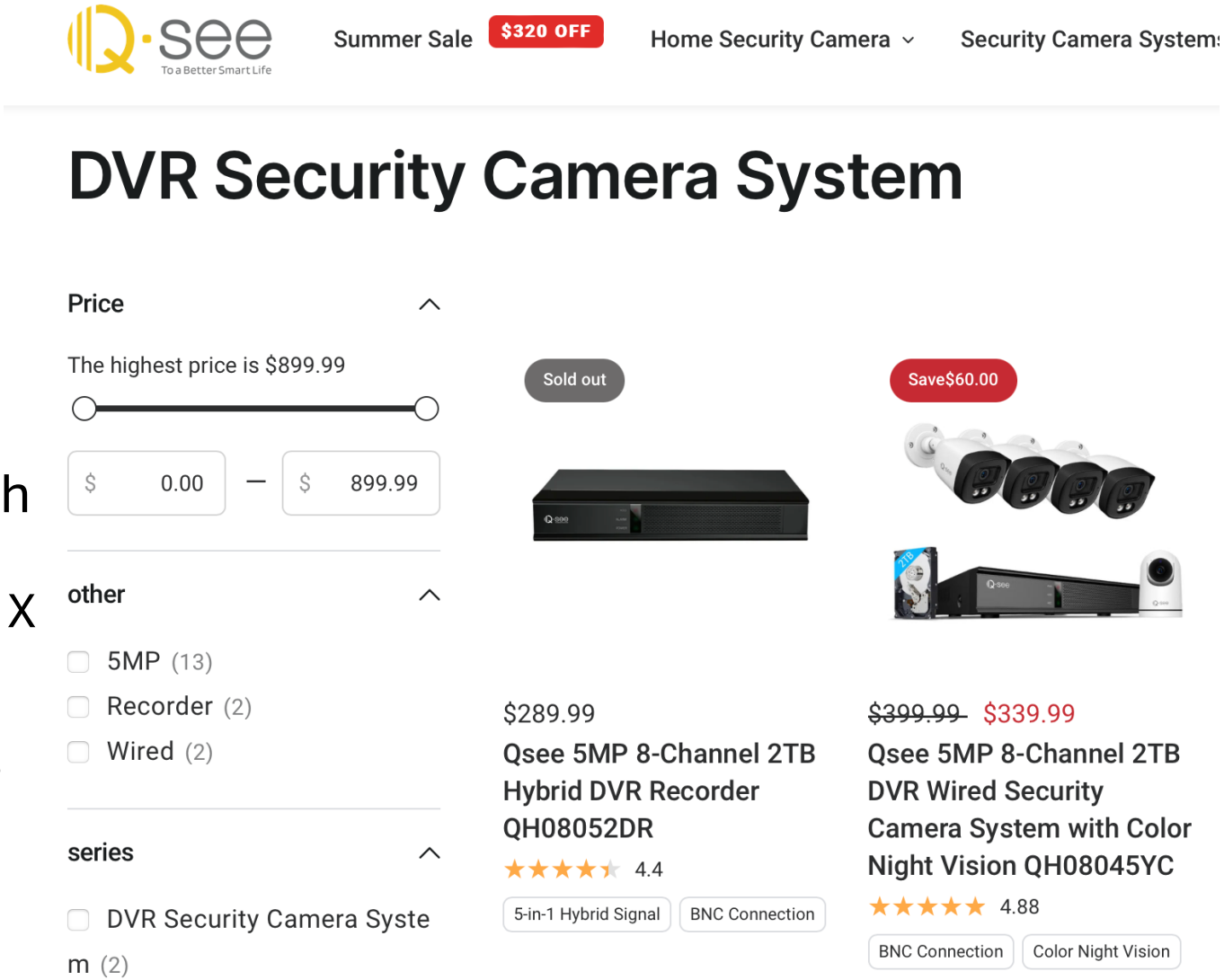
Some ARM Commands

- MOV, ADD, SUB,… similar to x64
  - No push or pop.
- LDR : load from memory
  - LDP load 2 registers first then second, same as two calls to LDR
  - ADR: load the address of some memory (e.g., a program constant).
- STR: Store into memory
  - STP: store 2 registers first then second, same as two calls to STR
- B label: Branch, jumps to label, (like JMP in x64)
- CMP: compare two registers,
- BZ, BEQ, BNQ, BGT, BLT, branch based on results of last compare.
- CBZ register, label: Jump to label if register is zero.

…. Google for other commands.

Backdoor

- Example Firmware analysis from my own research.
- This is ARM7 (not AArch 64).
  - E.g., R for registers not X
- The log in function has hardwired password!



Summary

- ARM AArch 8: is the standard on MAC, and coming soon to Windows laptops.

- More recent, better designed than x64.
  - Fewer memory accesses.
  - Less management on function calls.

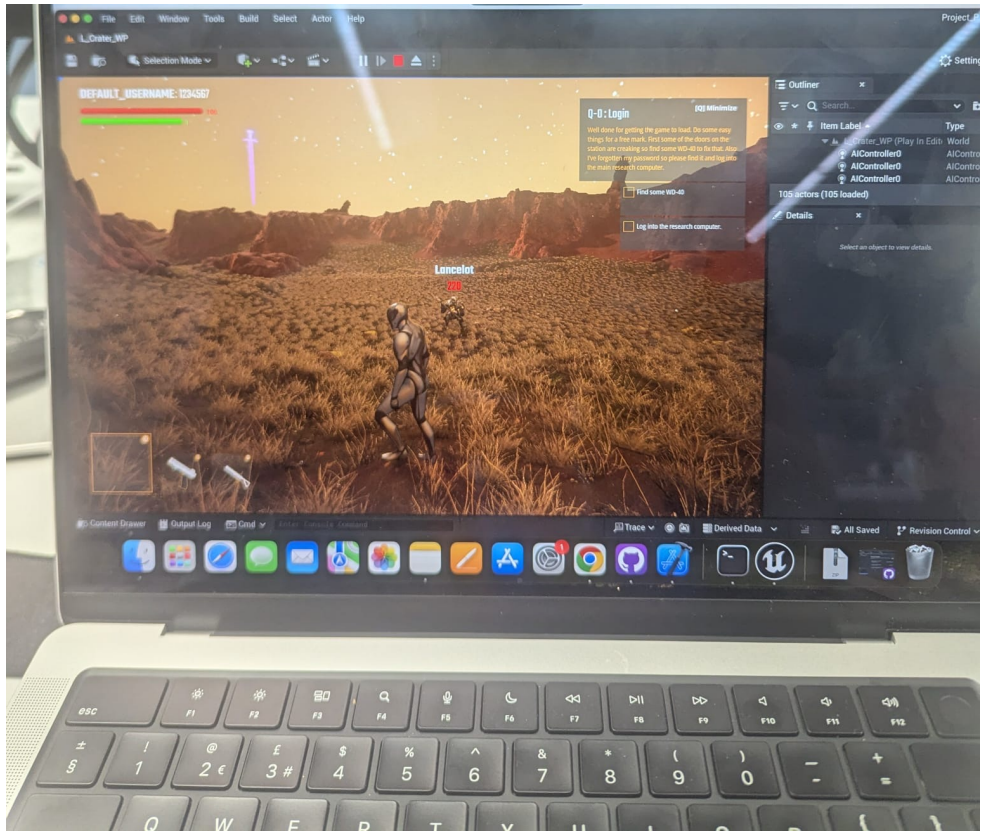
- Most likely you will be working on ARM machines at some point in your career.

- Mac users will need it for the exercises. It may be on the exam.

The Exercise Game

The module CA

- The CA for the first half of the module will take the form of a game.
- You run this on your own laptop, Windows, Linux or Mac, and the game connects to our servers.
- To get the CA marks you need to complete quests while connected to the Internet.
  - Your marks will automatically appear in Canvas.



The Quests

- The quests will appear on the game map when they are released, with additional instructions on Canvas.
  - However, you don’t have to wait for this, you could do all the quests today!

- To complete the quests, you will have to hack the game using what I teach you in lectures.
  - The quests might seem completely impossible until after the relevant lecture.

- After the formal deadlines, I will go through the solutions to the exercise in detail, and you can still complete the quests for half marks.

Important

- Download the game from the Canvas page, make sure you can run it and connect to the game server.
- Complete Quest 0. Simple in game task. Free Marks!
- This is experimental. Let as know asap if there are problems.
- Hacking the exercises, and completely them in other ways is fine.
  - This will be harder than completing them by the intended method.
- You must not submit flags for other students. The game collected anonymized machine fingerprints. Do not use the same machine as another student. Tell us asap if this is an issue.
- Disrupting the game for others is not allowed.

Next week: Buffer overflow attacks and defense.